LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 7 – Abstract & Interface



NAMA : ISNA FARHAH UMAMI

NIM : 103132430004

KELAS : PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Abstraksi (Abstraction) dan Antarmuka (Interface) merupakan pilar utama dalam pemrograman berorientasi objek (OOP) yang digunakan untuk menyembunyikan detail kompleksitas internal serta menyediakan struktur umum bagi kelas-kelas pengimplementasinya.

- Kelas Abstrak (Abstract Class)

  Kelas abstrak dideklarasikan menggunakan kata kunci abstract. Kelas ini bertindak sebagai kerangka tingkat tinggi yang tidak dapat diinstansiasi secara langsung menjadi objek. Kelas abstrak dapat memuat atribut (field), metode konkret (metode yang memiliki badan/implementasi), serta metode abstrak (metode tanpa badan yang wajib di-override oleh subclass konkret). Kelas abstrak sangat cocok digunakan ketika beberapa kelas turunan memiliki hubungan keluarga dekat (satu silsilah hierarki) dan berbagi atribut serta perilaku dasar yang sama.
- Antarmuka (Interface)

  Interface adalah sebuah kontrak perilaku (behavioral contract) publik yang mendefinisikan himpunan metode yang harus dipenuhi oleh suatu kelas. Berbeda dengan kelas yang hanya mendukung single inheritance, Java mengizinkan sebuah kelas mengimplementasikan banyak interface sekaligus (multiple inheritance implisit). Sejak versi Java 8+, interface memiliki kapabilitas baru yaitu:
  a. Default Method: Metode di dalam interface yang memiliki implementasi default menggunakan kata kunci default, sehingga kelas implementasi tidak dipaksa untuk melakukan override.
  b. Static Method: Metode utilitas terikat pada interface yang dipanggil langsung melalui nama interface tanpa instansiasi objek.
- Polimorfisme Heterogen

  Polimorfisme memfasilitasi penanganan seragam terhadap berbagai objek yang berbeda tipe melalui antarmuka atau kelas induk yang sama. Dalam konteks koleksi data statis (array), polimorfisme heterogen memungkinkan array bertipe interface menyimpan objek-objek dari silsilah kelas yang sepenuhnya tidak sekerabat (seperti Tiket Bioskop dan Makanan), selama seluruh kelas tersebut mengimplementasikan interface yang sama.

B. Guided

1. Guided 1 - Abstract Class

a. Employee.java

```java
package com.isna.modul07.guided.g1;

//fondasi umum:belum punya peran spesifik, jadi tidak
boleh diinstansiasi langsung
public abstract class Employee {
    private String name;

    public Employee(String name){
        this.name = name;
    }

    public String getName(){
        return name;
    }

    //setiap pegawai pasti punya gaji, tapi rumusnya
beda - beda
    public abstract double earnings();
}
```

b. SalariedEmployee.java

```java
package com.isna.modul07.guided.g1;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double
weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
```

```
    public double earnings() {
        return weeklySalary;
    }
}
```

c. Main.java

```
package com.isna.modul07.guided.g1;

public class Main {
    public static void main(String[] args) {
        SalariedEmployee salaried = new
SalariedEmployee("Budi", 5000000);
        System.out.println("Nama: " +
salaried.getName());
        System.out.println("Gaji: Rp" +
salaried.earnings());
    }
}
```

Output :

```
PS D:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>  &
'D:\SEMESTER 3\Infrastruktur dan Platform untuk Sains Data\java\bin\java.exe'
 '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\SEMESTER 5\PBO (Praktiku
m)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01\modul-07-abstract-dan-interface\pr
oject\src\target\classes' 'com.isna.modul07.guided.g1.Main'
Nama: Budi
Gaji: Rp5000000.0
```

2. Guided 2 - Interface implementation

a. Payable.java

```
package com.isna.modul07.guided.g2;

//Standar sistem: siapa pun yang mau ditagih oleh kasir
wajib mengimplementasikan interface ini
public interface Payable {
    double getPaymentAmount();
}
```

b. Invoice.java

```java
package com.isna.modul07.guided.g2;

//Benda mati non-pegawai, tapi tetap diikat oleh aturan
public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity,
double pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
    }

    public String getPartName() {
        return partName;
    }

    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }

}
```

c. Employee.java

```java
package com.isna.modul07.guided.g2;

//Fondasi umum: belum punya peran spesifik, jadi tidak
boleh diinstansiasi langsung
public abstract class Employee implements Payable {
```

```
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

d. Main.java

```
package com.isna.modul07.guided.g2;

public class Main {
    public static void main(String[] args) {
        Payable[] items = new Payable[2];

        items[0] = new Invoice("Monitor LED", 2,
1500000);
        items[1] = new SalariedEmployee("Budi",
5000000);

        // Dynamic dispacth: Java otomatis mengeksekusi
rumus yang tepat sesuai objek
        for (Payable item : items) {
            System.out.println("jumlah yang perlu
dibayar: Rp" + item.getPaymentAmount());
        }
```

```
    }


}
```

e. SalariedEmployee

```
package com.isna.modul07.guided.g2;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double
weeklySalary) {
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

Output :

```
PS D:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\praktikum
-pbo-ps1ds-05-reg-01>  d:; cd 'd:\SEMESTER 5\PBO (Pr
aktikum)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01'; &
 'D:\SEMESTER 3\Infrastruktur dan Platform untuk Sai
ns Data\java\bin\java.exe' '-XX:+ShowCodeDetailsInEx
ceptionMessages' '-cp' 'D:\SEMESTER 5\PBO (Praktikum
)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01\modul-07-a
bstract-dan-interface\project\src\target\classes' 'c
om.isna.modul07.guided.g2.Main'
jumlah yang perlu dibayar: Rp3000000.0
jumlah yang perlu dibayar: Rp5000000.0
```

3. Guided 3 - Interface Java 8+ (Default & Static Method)

   a. Employee.java

```java
package com.isna.modul07.guided.g3;

public abstract class Employee implements Payable {
    private String name;

    public Employee(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public abstract double earnings();

    @Override
    public double getPaymentAmount() {
        return earnings();
    }
}
```

   b. Invoice.java

```java
package com.isna.modul07.guided.g3;

public class Invoice implements Payable {
    private String partName;
    private int quantity;
    private double pricePerItem;

    public Invoice(String partName, int quantity, double
pricePerItem) {
        this.partName = partName;
        this.quantity = quantity;
        this.pricePerItem = pricePerItem;
```

```
    }

    public String getPartName() {
        return partName;
    }

    //Hitung tagihan
    @Override
    public double getPaymentAmount() {
        return quantity * pricePerItem;
    }
}
```

c. Payable.java

```
package com.isna.modul07.guided.g3;

public interface Payable {
    double getPaymentAmount();

    //fitur baru
    default void printRecipt(){
        System.out.println(x: "Struk Resmi - Total: Rp" +
getPaymentAmount());
    }

    //fungsi utilitas
    static void logTransaction(String message) {
        System.out.println("[AUDIT LOG] " + message);
    }

}
```

d. SalariedEmployee.java

```java
package com.isna.modul07.guided.g3;

public class SalariedEmployee extends Employee {
    private double weeklySalary;

    public SalariedEmployee(String name, double weeklySalary)
{
        super(name);
        this.weeklySalary = weeklySalary;
    }

    @Override
    public double earnings() {
        return weeklySalary;
    }
}
```

e. Main.java

```java
package com.isna.modul07.guided.g3;

public class Main {
    public static void main(String[] args) {
        Payable.logTransaction("Sistem pembayaran
diinisialisasi.");

        Payable[] items = new Payable[2];
        items[0] = new Invoice("Ram DOR 5 16GB", 2, 2000000);
        items[1] = new SalariedEmployee("Budi", 5000000);

        for (Payable item : items) {
            item.printReceipt();
        }

        Payable.logTransaction("Transaksi selesai
diproses.");
```

```
    }
}
```

Output :

```
PS D:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\praktikum-pbo-ps1ds
-05-reg-01>  d:; cd 'd:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\p
raktikum-pbo-ps1ds-05-reg-01'; & 'D:\SEMESTER 3\Infrastruktur
dan Platform untuk Sains Data\java\bin\java.exe' '-XX:+ShowCod
eDetailsInExceptionMessages' '-cp' 'D:\SEMESTER 5\PBO (Praktik
um)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01\modul-07-abstract-
dan-interface\project\src\target\classes' 'com.isna.modul07.gu
ided.g3.Main'
[AUDIT LOG] Sistem pembayaran diinisialisasi.
Struk Resmi - Total: Rp4000000.0
Struk Resmi - Total: Rp5000000.0
[AUDIT LOG] Transaksi selesai diproses.
```

C. Unguided

1. Soal Unguided

   Membuat sistem kasir bioskop terpadu yang memproses pembayaran heterogen (Tiket Reguler, Tiket VIP, dan Paket Snack F&B) menggunakan kontrak interface ItemTransaksi, kelas abstrak TiketBioskop, serta default & static method Java 8+.

   Jawaban :

   a. ItemTransaksi.java

```java
package com.isna.modul07.unguided;

public interface ItemTransaksi {
    double getNominalBayar();
    String getNamaTransaksi();

    default void cetakRincianItem() {
        System.out.println("- " + getNamaTransaksi() + " |
Total: Rp" + getNominalBayar());
    }

    static void catatLogKasir(String pesan) {
        System.out.println("[LOG SISTEM KASIR] " + pesan);
```

```
    }
}
```

b. TiketBioskop.java

```
package com.isna.modul07.unguided;

public abstract class TiketBioskop implements ItemTransaksi {
    private String judulFilm;
    private double hargaDasar;
    private int jumlahKursi;

    public TiketBioskop(String judulFilm, double hargaDasar,
int jumlahKursi) {
        this.judulFilm = judulFilm;
        this.hargaDasar = hargaDasar;
        this.jumlahKursi = jumlahKursi;
    }

    public String getJudulFilm() {
        return this.judulFilm;
    }

    public double getHargaDasar() {
        return this.hargaDasar;
    }

    public int getJumlahKursi() {
        return this.jumlahKursi;
    }

    public abstract double hitungTotalBayar();
    public abstract String getNamaTransaksi();

    @Override
    public double getNominalBayar() {
        return hitungTotalBayar();
    }
}
```

c. TiketReguler.java

```
package com.isna.modul07.unguided;

public class TiketReguler extends TiketBioskop {
    private int nomorStudio;

    public TiketReguler(String judulFilm, double hargaDasar,
int jumlahKursi, int nomorStudio) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.nomorStudio = nomorStudio;
    }

    public int getNomorStudio() {
        return this.nomorStudio;
    }

    @Override
    public double hitungTotalBayar() {
        return getHargaDasar() * getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket Reguler (Studio " + this.nomorStudio +
") - "
                + getJudulFilm() + " (" + getJumlahKursi() +
" x Rp" + getHargaDasar() + ")";
    }
}
```

d. TiketVIP.java

```
package com.isna.modul07.unguided;

public class TiketVIP extends TiketBioskop {
    private double biayaFasilitasVIP;

    public TiketVIP(String judulFilm, double hargaDasar, int
```

```
jumlahKursi, double biayaFasilitasVIP) {
        super(judulFilm, hargaDasar, jumlahKursi);
        this.biayaFasilitasVIP = biayaFasilitasVIP;
    }

    public double getBiayaFasilitasVIP() {
        return this.biayaFasilitasVIP;
    }

    @Override
    public double hitungTotalBayar() {
        return (getHargaDasar() + this.biayaFasilitasVIP) *
getJumlahKursi();
    }

    @Override
    public String getNamaTransaksi() {
        return "Tiket VIP Premiere - " + getJudulFilm() + "
("
                + getJumlahKursi() + " x Rp" +
(getHargaDasar() + this.biayaFasilitasVIP) + ")";
    }
}
```

e. PaketSnack.java

```
package com.isna.modul07.unguided;

public class PaketSnack implements ItemTransaksi {
    private String namaPaket;
    private int jumlahPorsi;
    private double hargaPerPorsi;

    public PaketSnack(String namaPaket, int jumlahPorsi,
double hargaPerPorsi) {
        this.namaPaket = namaPaket;
        this.jumlahPorsi = jumlahPorsi;
        this.hargaPerPorsi = hargaPerPorsi;
    }
```

```
    public String getNamaPaket() {
        return this.namaPaket;
    }

    public int getJumlahPorsi() {
        return this.jumlahPorsi;
    }

    public double getHargaPerPorsi() {
        return this.hargaPerPorsi;
    }

    @Override
    public double getNominalBayar() {
        return this.jumlahPorsi * this.hargaPerPorsi;
    }

    @Override
    public String getNamaTransaksi() {
        return "F&B Snack: " + this.namaPaket + " (" +
this.jumlahPorsi + " x Rp" + this.hargaPerPorsi + ")";
    }
}
```

f. Main.java

```
package com.isna.modul07.unguided;

public class Main {
    public static void main(String[] args) {
        ItemTransaksi.catatLogKasir("Membuka transaksi baru
untuk Pelanggan #042.");

        TiketReguler tiketReg = new
TiketReguler("Interstellar", 40000.0, 2, 3);
        TiketVIP tiketVip = new TiketVIP("Dune: Part Two",
60000.0, 2, 25000.0);
```

```
        PaketSnack snack = new PaketSnack("Combo Popcorn
Caramel + 2 Soda", 1, 55000.0);

        ItemTransaksi[] daftarBelanja = new ItemTransaksi[] {
            tiketReg,
            tiketVip,
            snack
        };

        double totalTransaksi = 0.0;

        for (ItemTransaksi item : daftarBelanja) {
            item.cetakRincianItem();
            totalTransaksi += item.getNominalBayar();
        }

        System.out.println("TOTAL TRANSAKSI : Rp" +
totalTransaksi);
        ItemTransaksi.catatLogKasir("Transaksi #042 selesai
diproses.");
    }
}
```

Output :

```
PS D:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01>  d:; cd 'd:\SEME
STER 5\PBO (Praktikum)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01'; & 'D:\SEMESTER 3\Infrastr
uktur dan Platform untuk Sains Data\java\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMes
sages' '-cp' 'D:\SEMESTER 5\PBO (Praktikum)\PRAKTIKUM\praktikum-pbo-ps1ds-05-reg-01\modul-
07-abstract-dan-interface\project\src\target\classes' 'com.isna.modul07.unguided.Main'
[LOG SISTEM KASIR] Membuka transaksi baru untuk Pelanggan #042.
- Tiket Reguler (Studio 3) - Interstellar (2 x Rp40000.0) | Total: Rp80000.0
- Tiket VIP Premiere - Dune: Part Two (2 x Rp85000.0) | Total: Rp170000.0
- F&B Snack: Combo Popcorn Caramel + 2 Soda (1 x Rp55000.0) | Total: Rp55000.0
TOTAL TRANSAKSI : Rp305000.0
[LOG SISTEM KASIR] Transaksi #042 selesai diproses.
```

Penjelasan :

Program kasir bioskop terpadu ini menerapkan kombinasi kelas abstrak, interface, serta polimorfisme heterogen. Antarmuka ItemTransaksi berfungsi sebagai kontrak publik utama dengan dua metode abstrak (getNominalBayar() dan getNamaTransaksi()), metode default (cetakRincianItem()) untuk pencetakan baris struk, serta metode statis (catatLogKasir()) untuk fungsi pembantu log. Kelas abstrak TiketBioskop mengimplementasikan ItemTransaksi dan menurunkan properti dasar

ke TiketReguler serta TiketVIP. Sementara itu, PaketSnack mengimplementasikan ItemTransaksi secara independen. Di dalam Main.java, objek-objek heterogen disatukan dalam array statis ItemTransaksi[] dan diproses menggunakan perulangan secara otomatis.

D. Kesimpulan

Berdasarkan praktikum Modul 7 tentang Abstraksi, Kontrak Interface, dan Polimorfisme Heterogen, dapat disimpulkan bahwa:

1. Kelas abstrak (abstract class) sangat tepat digunakan untuk mendefinisikan kerangka dasar serta perilaku bersama dari silsilah kelas yang sekeluarga, namun tidak diizinkan untuk diinstansiasi secara langsung menjadi objek.
2. Antarmuka (interface) bertindak sebagai kontrak publik yang mampu menyatukan berbagai kelas dari hierarki yang sepenuhnya berbeda ke dalam satu tipe referensi yang seragam.
3. Dukungan Default Method dan Static Method sejak Java 8+ memungkinkan penambahan metode umum beserta fungsi utilitas ke dalam interface tanpa merusak kompabilitas pada kelas-kelas implementasinya.
4. Polimorfisme heterogen mempermudah pemrosesan berbagai objek berbeda tipe dalam satu wadah koleksi (seperti array), sehingga iterasi data dapat dilakukan dengan lebih bersih, terstruktur, dan efisien.